

Module 2: Advanced Data Structures & Graph Algorithms

→ Self balancing tree! — AVL tree (Height Balanced)

- 1 node below root is left & right

- Subtree have the same height

- Balanced binary tree

- Every node is either left or right

- If B is left and right subtree then B is on right

is

(a) B and B₂ are over trees

(b) B₁ - B₂ / 5

where B₁ and B₂ are heights of B and B₂

subtree

* Balance factor: —

To implement in AVL tree each node must contain

a balance factor which indicates its state of

balance relative to its subtrees

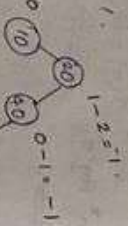
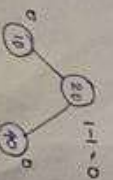
ie,

Height of left subtree — Height of right subtree

Height of left subtree — Height of right subtree

The balance factor in a balanced tree can have

values from -1, 0, or 1



balanced

Balance factor B_i is 0 at root -1 0 1 then the tree is unbalanced.



unbalanced

why are we?

- most of the binary search tree operation (eg insert, deletion, etc) take O(n) time where

n is the height of tree

- Minimum height of tree is at least a log

- Height of tree is always O(log n) where n

is the number of nodes in the tree

- Time complexity of all AVL tree operations are

O(log n)

- Maximum number of nodes in AVL tree of height h

h - 2^{h-1}

- Minimum number of nodes in AVL tree of height h

is given by,

N(h) = N(h-1) + N(h-2) + 1

N(0) = 1

N(1) = 2

N(2) = 4

N(3) = 7

N(4) = 12

N(5) = 21

N(6) = 37

N(7) = 64

$$N(1) = 1 + N(2) + N(3)$$

$$N(2) = 1 + N(1) + N(3)$$

$$= 1 + 1 + 2$$

$$= 4$$

Find min/max height of tree with 7 nodes. Also find the height of a tree with single node is 0.

$$N(3) = 1 + 1 + 2$$

$$= 4$$

$$N(4) = 1 + 1 + 4$$

$$= 6$$

$$N(5) = 1 + 12 + 7$$

$$= 20$$

Find maximum height of tree with 7 nodes.

$$H_0 = H_{n-1} + H_{n-2} + 1$$

$$H_2 = 1 + 2 + 1 = 4$$

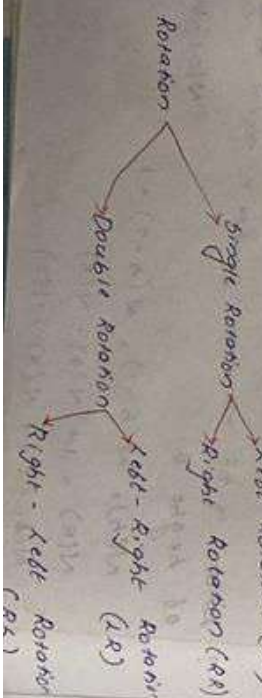
$$H_3 = 4 + 2 + 1 = 7$$

So also 5-3-5 nodes of 7 nodes.

* AVL Rotation :-

- Because a tree has a procedure called AVL rotation.

- Rotation is the process of moving the nodes to either left or right to make tree balanced.



→ Single Left Rotation (LL Rotation) :-

- Every node moves one position to left from its current position.

Right Right insertion → rotate left

E.g. insert 4, 5, 3



unbalanced tree made in form of LL rotation which moves nodes one position left

→ Single Right Rotation (RR Rotation) :-

- Every node moves one position to right from the current position.

- Left Left insertion → rotate right

E.g. insert 3, 2, 1

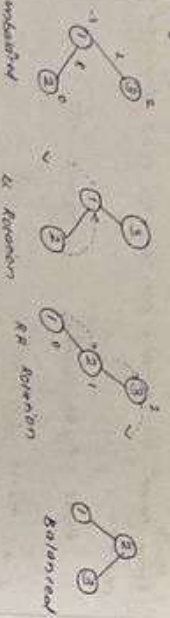


unbalanced in RR rotation moves node one position to right

→ Left Right Rotation (LR Rotation) :-

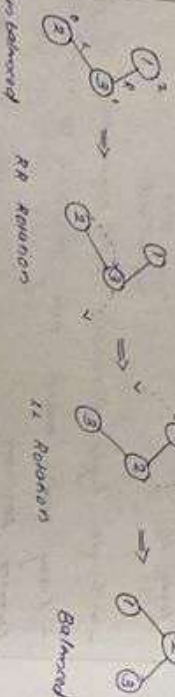
- combination of single left rotation followed by single right rotation.

E.g. 8, 4, 2

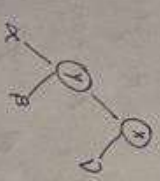
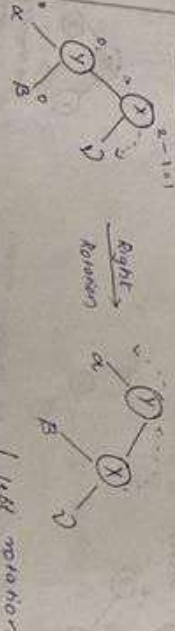


→ Right Left Rotation (RL Rotation) —
- combination of single right rotation followed by single left rotation.

E.g. insert 1, 4, 2



Example



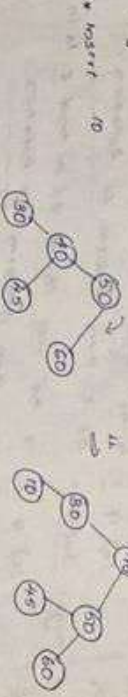
20. Balance an unbalanced tree —

1. Insert node in to B.T
2. compute balance factors
3. Decide the pivot node

After step 3, examine whether any node's balance factor is different from 1 or -1. If so tree is unbalanced. The node which is different is B.T. is called pivot node.

4. Balance the unbalanced tree

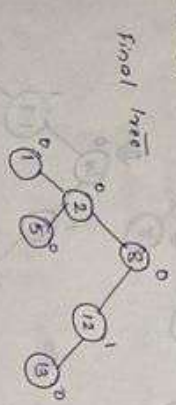
E.g. 50, 40, 30, 45, 60



E.g. 50, 40, 60, 55, 70



Example



Final tree

→ AVL tree insertion: —

1. Insert new node as the leaf node
2. After insertion check the balance factor of every node
3. If tree is unbalanced perform the suitable rotation
4. If $BF \leq 1$ or $BF \geq 1$ then no rotation is needed
5. If $BF \leq 2$ or $BF \geq 2$ then perform the suitable rotation

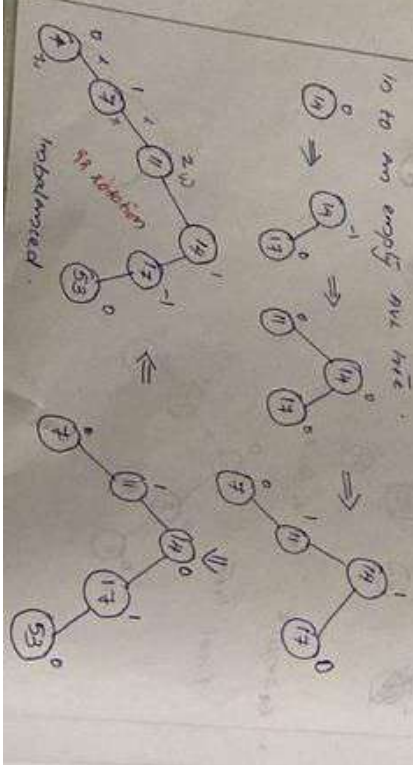
unbalanced node on the path from root to node x is the node x or its parent

child node y is the grandchild of x if x is the parent of y

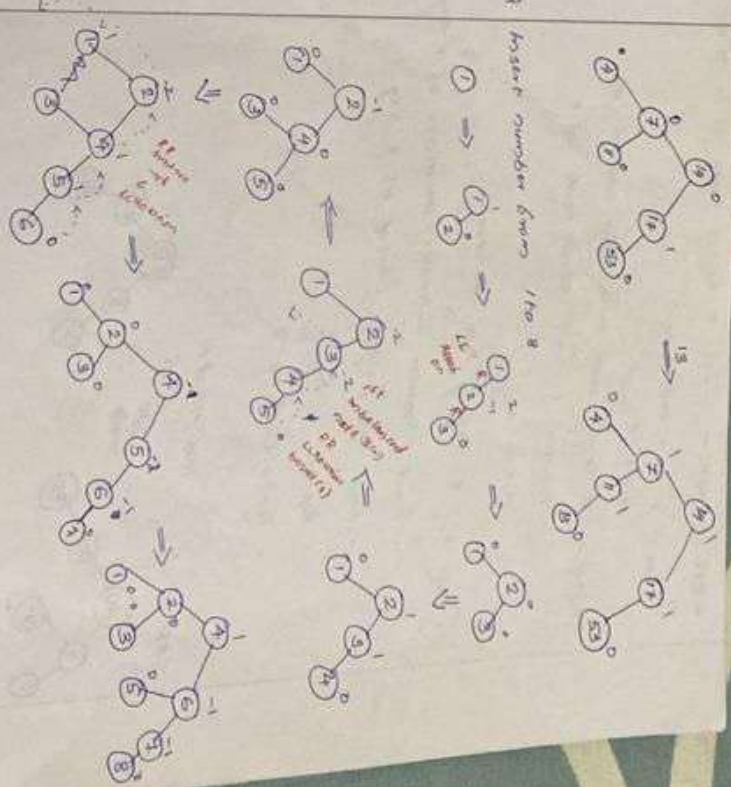
from x to root

Time complexity of AVL tree insertion $O(\log n)$

Q. Create an AVL tree with elements 14, 17, 11, 7, 53, 4, 13



Q. Insert number from 1 to 8



→ Operations on an AVL tree: —

1. Insert/delete new element into tree using BST
2. Check balance factor
3. If balance factor of every node is more than 1, then perform suitable rotation to make balanced.

$$- (13) p, 190196a = 2$$

6. X & Y

1. 16 balance for 0

2. 15

X = right child

Y = left child

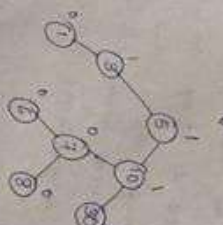
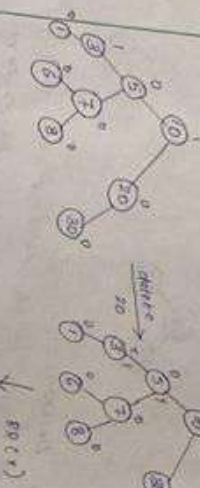
3. If Y is the right child of X and X is the left child of Y then perform RR rotation at R-T & X

4. If Y is the left child of X and X is the right child of Y then perform LL rotation at R-T & X

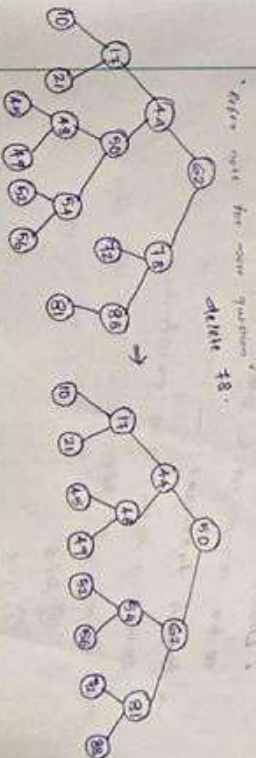
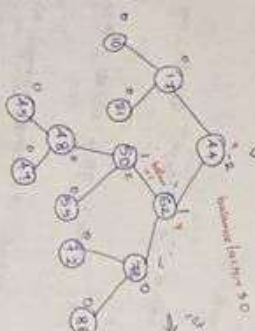
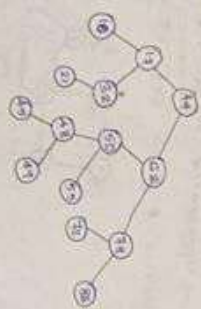
5. If Y is the right child of X and X is the right child of Y then perform RL rotation at R-T & X

6. If Y is the left child of X and X is the left child of Y then perform RL rotation at R-T & X

8. Delete 20 from given AVL tree

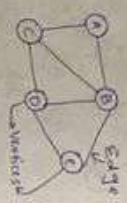
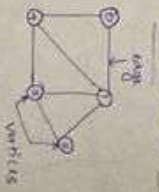


DATE 32



→ Graphs:-

- A graph G is represented as $G=(V,E)$ where
 $V \rightarrow$ Set of vertices + $E \rightarrow$ Set of edges



* Types of graphs:-

- Undirected graph: A graph with only undirected edge
- Directed graph: Graph only with directed edges
- Directed acyclic graph: directed graph with no cycles

- Cyclic graph: directed graph with at least one cycle
- Weighted graph: Each edge is assigned a weight
- Disconnected graph: vertices in a graph do not need to be connected to other vertices

→ Graph traversal:-

- Something written in a graph

- ① BFS
- ② DFS



→ Breadth First Search (BFS):-

- Use Queue data structure for implementation

Steps:

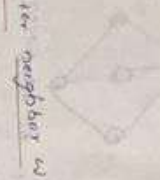
1. Define a Queue of size total number of vertices in the graph
2. Since any vertex is starting point for traversal visit that vertex + insert it into the queue
3. Visit all the unvisited adjacent vertices of the vertex that is as visited and insert it into the queue
4. If no adjacent vertex is found, then remove it from the queue
5. Repeat step 3 and 4 until queue becomes empty

* Algorithm:-

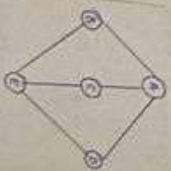
Algorithm BFS (G, n)

1. Set all nodes are unvisited
2. Mark the starting vertex as visited and put it into an empty queue Q
3. While Q is not empty
 1. Dequeue x from Q
 2. While x has an unvisited neighbor w
 1. Mark w as visited
 2. Enqueue w into Q
4. If while is may unvisited node x
 1. Visit x and insert it into Q both steps

Visit order →
 BFS order

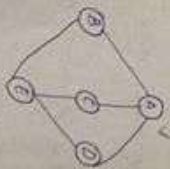


Example 1 -



- Let the strategy now be to insert in queue

Node visited: A



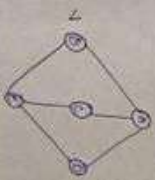
Traversal: A

- Dequeue A insert the adjacent unvisited vertex

in Q

Node visited: AB

Traversal: ABCD



Traversal: ABCD

insert the adjacent unvisited vertex

Node visited: ABC

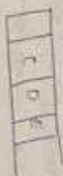


insert next adjacent unvisited vertex

Node visited: ABCD

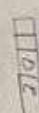


Now no unvisited adjacent vertex for A.
Remove B and insert its adjacent vertex in queue
Node visited: ABCDE



Traversal: ABCDE

Remove C adjacent vertex of C is visited. No new vertex



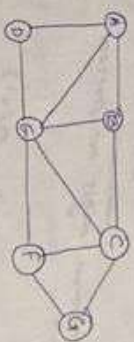
Remove D

Remove E

Now queue empty

BFS traversal: A B C D E

Q1

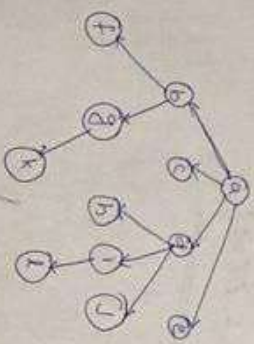


BFS: A D E B C F G

Q2



Traversal: A B C D E



a

abcde

cdefg

defg

efghij

fghij

ghij

hijk

[abcde]

[cdefg]

[defg]

[efghij]

[fghij]

[ghij]

[hijk]

BFS algorithm complexity:-

- If the graph is represented as an adjacency list
- each vertex is enqueued and dequeued at most once.
- each queue operation takes $O(1)$ time.
- do the time needed to do queue operation is $O(V)$.
- the adjacency list of each vertex is scanned only when the vertex is dequeued.
- each adjacency list scanned at most once.
- sum of the lengths of all adjacency lists is $O(E)$.
- total time spent in scanning adjacency list is $O(E)$.
- time complexity of BFS = $O(V) + O(E)$.
- $O(V+E)$.

Dense graph: minimum edges not present

$$E = O(V^2)$$

$$\text{Complexity} = O(V) + O(V^2) = \underline{\underline{O(V^2)}}$$

Adjacency matrix

- V^2 entries. each checked once.

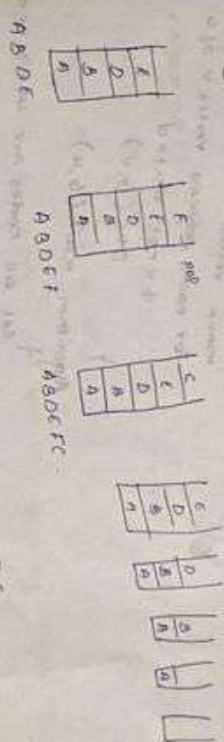
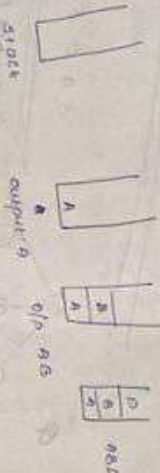
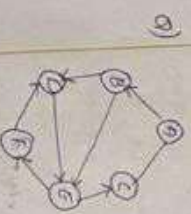
$$\underline{\underline{\text{Complexity} = O(V^2)}}$$

DFS First Search (DFS):-

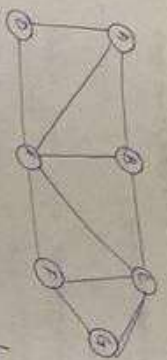
- Use stack data structure

Steps:

- 1) Define a stack of size total number of vertices in the graph.
- 2) Start any vertex as starting point for traversal. visit that vertex and push it on to the stack.
- 3) Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack.
- 4) Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.
- 5) When there is no adjacent vertex to visit the use backtracking & pop one vertex from the stack.
- 6) Repeat 3, 4, 5 until stack become empty.

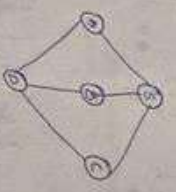


Traversal: A B C D E F



Inverse: ABCEDFG

S A D B C



Algorithm DFS(u)

mark u as visited
for each adjacent vertex v of u

if v not visited

DFS(v)

Algorithm main(u)

set all nodes as unvisited

DFS(u)

for any node x which is not given

DFS(x)

• DFS complexity :-

• Adjacency list: visited

• Each vertex at most once $O(V)$

• Visited stored at most once $O(E)$

- total $O(V+E)$

• Adjacency matrix :-

• $|V|^2$ entries in the matrix. Each entry checked

once. Time complexity $= O(V^2)$

→ Topological sorting :- [Topological ordering]

- Topological sorting is possible iff the graph is DAG (directed acyclic graph)

- It is linear ordering of its vertices such that for every directed graph edge (u,v) from vertex u to vertex v, u comes before v in the ordering.

- If the graph contains cycle, no linear ordering is possible.

- There may exist multiple different topological ordering for a given DAG.

• Algorithm :-

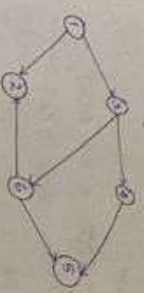
1. Identify a node with no incoming edges (indegree)

2. Add this node to the ordering

3. Remove this node and all its outgoing edges from the graph.

4. Repeat 1 to 3 until graph becomes empty

Q. write its topological sorting for the DAG.



→ Remove 1 and its associated edges.
topological ordering: 1



→ Remove 3 and its associated edges.
topological ordering: 1, 3



→ Remove 2 and its associated edges.
topological order: 1, 3, 2



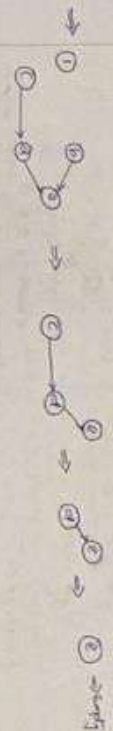
→ Remove 6 and its associated edges.
topological order: 1, 3, 2, 6

→ topological order: 1, 3, 2, 6, 4, 5
empty graph

Q.



order: a, b, c, d, e



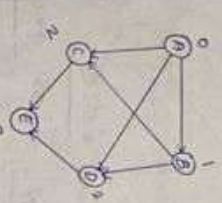
→ Remove a and its associated edges.
order: a, c, b, d, e



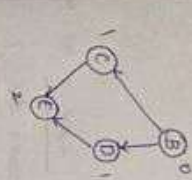
→ Remove a, c, d, b, e



Q.

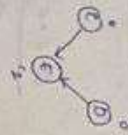


Remove a



order: a

Remove b



order: a, b

Case 1
Remove c



Order: a b c

Remove d

2

Order: a b c d

Empty Remove e

Empty

Order: a b c d e

* Complexity: -

Determine degree of each node - $O(V)$

Looking at each directed edge in the graph

once

Time to determine the nodes with no incoming edges $O(V)$

Step 1 complexity $O(V+V)$

Add nodes and we run out of nodes with no incoming edges. Repeat for every node $O(V)$

Total: $O(V+V)$

Case 2
Remove d



Order: a b c

Remove c

3

Order: a b c d

Empty Remove e

Empty

Order: a b c d e

→ Strongly Connected Components -

- It applies only to directed graphs

- A directed graph is strongly connected if there is a path from every vertex to every other vertex

- For every pair of vertices (u, v) there exists a path from u to v and v to u , both u and v are reachable from each other

- A directed graph is a maximal strongly connected subgraph

- Also form a partition of a graph

components

$SC_1: [a, b, c]$

$SC_2: [d, e]$

$SC_3: [f, g, h]$

$SC_4: [i, j, k]$

$SC_5: [l, m, n]$

$SC_6: [o, p, q]$

$SC_7: [r, s, t]$

$SC_8: [u, v, w]$

$SC_9: [x, y, z]$

$SC_{10}: [aa, bb, cc]$

$SC_{11}: [dd, ee, ff]$

$SC_{12}: [gg, hh, ii]$

$SC_{13}: [jj, kk, ll]$

$SC_{14}: [mm, nn, oo]$

$SC_{15}: [pp, qq, rr]$

$SC_{16}: [ss, tt, uu]$

$SC_{17}: [vv, ww, xx]$

$SC_{18}: [yy, zz, aa]$

$SC_{19}: [bb, cc, dd]$

$SC_{20}: [ee, ff, gg]$

$SC_{21}: [hh, ii, jj]$

→ Rosarino's algorithm: —

2 pass algorithm

(1-4-0) Post

5-7-5 pass 2

1. Set all vertices of graph G are connected
2. Create an empty stack S .

2 Canada on empty stack 3

3 Do Dis traveled to unvisited 4 sep
as visited 11 o water has no
unvisited neighbor
Push it into the 951048

4 Position the above stop until all wires are visible.

5. Reverse the graph (α^T)

5. Reverse the front

7. 206.11 5 is not empty

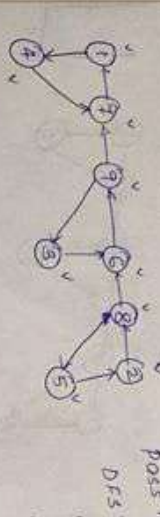
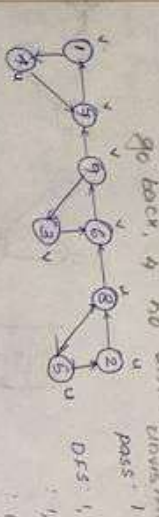
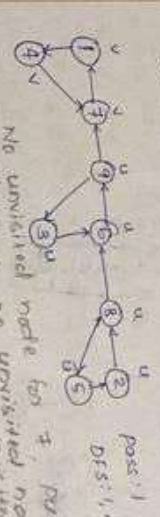
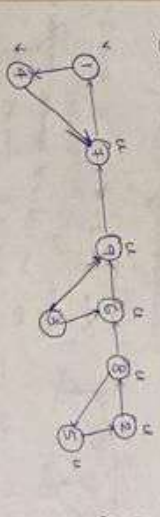
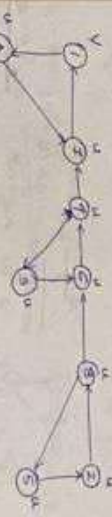
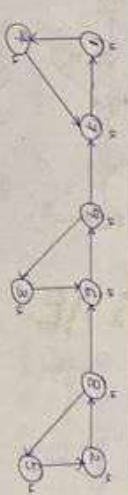
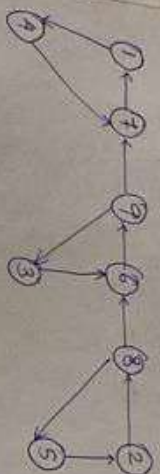
1. POP ONE vertex V .

2. 16 ✓ is not visited

1 set v' as visited

2. call DFS(V'). If call point see above

? find sec of the signal using the algo. showing each step



possit
disit

pass
DFS

poss: 1
DFS: 1, 4, 7

42	43	44	45
----	----	----	----

pass

 $DF5: 1, 4, 7, 10$

1,4,7,9,3
1,4,7,9,3,6

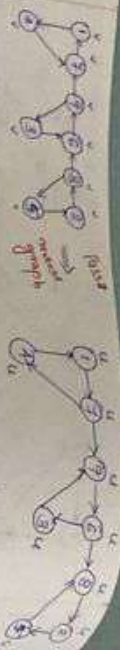
pass'

DFS: 1, 4, 7, 9, 5, 6, 8

1, 4, 7, 9, 3, 6, 8, 5
1, 4, 7, 9, 3, 6, 8, 5, 2

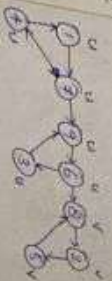
All nodes are visited

Completed

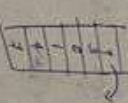
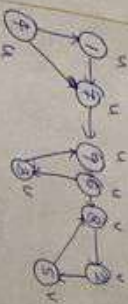


pop & from stack DFS normal based on

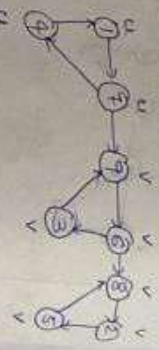
No unvisited neighbors
go back to parent
if no parent
go back



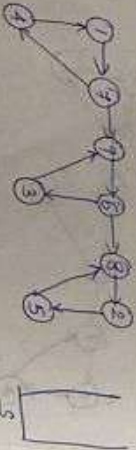
DFS: 1-2-3-4-5
pop 5 from stack
DFS normal based on



DFS: 1-2-3-4-5
pop 5 from stack
DFS normal based on



DFS: 1-2-3-4-5
pop 5 from stack
DFS normal based on



* Time complexity

1st pass DFS $O(V+E)$

Run graph $O(V+E)$

pass DFS $O(V+E)$

Total time $O(V+E)$

→ Disjoint set operation: -

Two or more sets with nothing in common are called disjoint sets

Eg: $S_1 = \{1, 2, 3, 4\}$ $S_2 = \{5, 6, 7\}$ $S_3 = \{8, 9\}$

Two sets S_1 and S_2 are said to be disjoint if $S_1 \cap S_2 = \emptyset$

The disjoint set data structure is also known as union-find datastructure and merge-find set

→ Disjoint set Representation: -

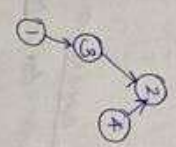
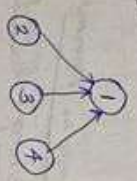
1. Linked list representation (tree representation)

2. Array representation

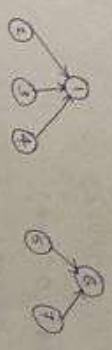
* Linked List Representation: -

$S_1 = \{1, 2, 3, 4\}$

This set can be represented as tree in different ways

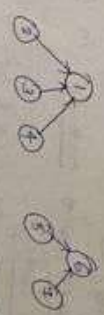


$S_1 = \{1, 2, 3, 4\}$ $S_2 = \{5, 6, 7\}$



Binary Representation:-

$S_1 = \{1, 2, 3, 4\}$ $S_2 = \{5, 6, 7\}$



	1	2	3	4	5	6	7
P	-1	1	1	1	6	-1	6

* Disjoint set operations:-

- make set
- find
- union

* make set:-

- make set (x) to will create a set that contains one element x

* find:-

- Determine which subset a particular element is in
- This can be used for determining if 2 elements are in same subset



• find (x) return 1, which is the root of the tree
tree that 3 belongs to which is the root of the tree

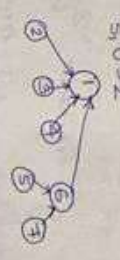
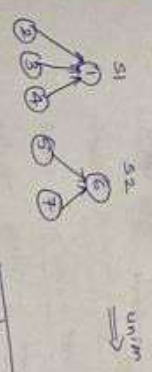
• find (x) return 6, which is the root of the tree
 that 6 belongs to

• union operation:-
 find (x) return 1, all nodes
 { 1, 2, 3, 4 }
 find (y) return 6, all nodes
 { 5, 6, 7 }
 return { 1, 2, 3, 4, 5, 6, 7 }

return { 1, 2, 3, 4, 5, 6, 7 }

* Union operation:- to one single subset in to two subsets belongs

- join 2 subsets in to one subset one perform union
 - as we have to check if the two subsets belong to same set or not then



	1	2	3	4	5	6	7
P	-1	1	1	1	6	1	6

Applications of disjoint set

- Used to merge 2 sets into one
- Check whether given elements are belongs to same set

union (1, 2)
 {
 x = find(1)
 y = find(2)
 if x != y then
 P[y] = x
 }